Project Report: Real-Time Network Traffic Visualization and Protocol Analysis

G Liktih – 2420030056 | J Hemanth – 2420039805
Computer Networks – Section 11

1. Project Overview

This project is designed to provide a real-time, graphical representation of network traffic within a local environment. The primary goal is to demonstrate how data 'packets' move between a client machine and various global servers. By using Kali Linux as the primary operating system and EtherApe as the visualization engine, the project transforms numerical network data into a dynamic topology map. This allows for the immediate identification of active protocols, bandwidth consumption, and server-client relationships.

2. System Architecture and Environment

The project is built on a tiered architecture involving a host operating system, a virtualization layer, and a guest operating system. This structure ensures that the network analysis can be performed in a controlled, sandboxed environment without disrupting the host system's primary functions.

- Components:
- Host OS: Windows (Physical Laptop)
- Hypervisor: Oracle VM VirtualBox
- Guest OS: Kali Linux (Version 2025.x)
- Visualization Tool: EtherApe

3. Network Design and Configuration

A critical phase of the project design is the network interface configuration. To allow the monitoring tool to 'see' the real network, the standard virtualized network isolation (NAT) was modified.

3.1 Bridged Adapter Mode

In Bridged Mode, the VirtualBox hypervisor bypasses the host's networking stack and connects the guest OS directly to the physical network card. This allows the Kali Linux VM to appear as a physical computer on the local Wi-Fi, receiving its own unique IP address (e.g., 10.250.21.79) from the router.

3.2 Promiscuous Mode

By setting the Promiscuous Mode to 'Allow All', the network card is instructed to capture every packet it detects on the medium, not just the ones addressed specifically to its MAC address. This is essential for full network visibility.

4. Implementation and Technical Execution

The following steps and commands were utilized to set up the environment and execute the visualization.

4.1 Tool Installation

```
sudo apt update && sudo apt install etherape -y
```

4.2 Executing the Monitor

To launch the tool with administrative privileges required for packet capture:

```
sudo etherape
```

4.3 Implementing Capture Filters

Initial execution resulted in a high-noise environment (the 'Yellow Ring' effect) due to background protocols like mDNS and ARP. To isolate web-based traffic and reduce system lag on older hardware, a Berkeley Packet Filter (BPF) was applied to the command line:

```
sudo etherape -f "port 80 or port 443"
```

5. Project Structure and Data Flow

The project follows a linear data flow model from packet capture to graphical rendering:

1. Packet Interception: The network interface (eth0/wlan0) captures incoming and outgoing frames.
2. Protocol Parsing: EtherApe analyzes the headers to identify source/destination IPs and port numbers.
3. Filtering: The BPF filter discards all non-web traffic (traffic not on port 80/443).
4. Mapping: The tool generates a node for each IP address and a line for each TCP/IP stream.
5. Visualization: Real-time rendering of line thickness based on throughput.

6. Traffic Analysis and Results

The filtered visualization results in a clean map with the following characteristics:

- Central Hub: The local Kali machine (10.250.21.79) acts as the primary focal point.
- External Nodes: Web servers (e.g., 149.154.x.x) appear on the periphery.

- Protocol Identifiers: Red lines represent HTTP, while Dark Red/Maroon lines represent HTTPS (encrypted) traffic.
- Dynamic Scaling: As a website is loaded in Firefox, the lines pulse and thicken to show data being pulled into the system.

7. Conclusion

This project successfully visualizes the 'hidden' layer of network communication. By configuring a virtual environment to interact directly with a physical network and applying targeted filters, we have created a high-impact, lightweight monitoring system. This design proves that network traffic can be audited and understood without the need for high-end computing resources, making it a viable tool for network troubleshooting and educational demonstrations.